

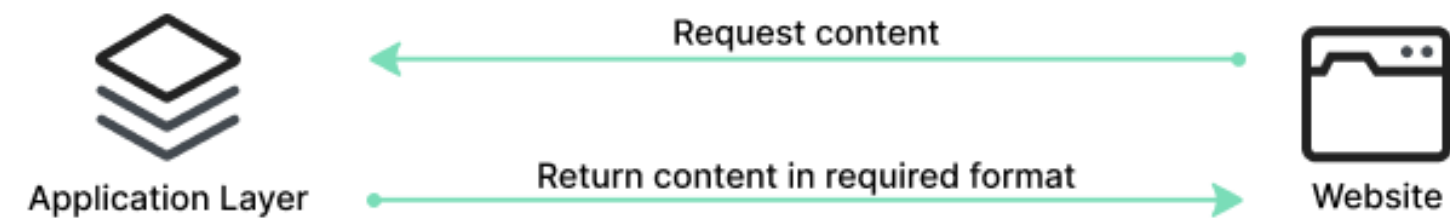


# Reliable Data Transfer and TCP Flow Control

**Join the lecture online on your dashboard**

# Layers recap

## Application Layer



## Presentation Layer



## Session Layer



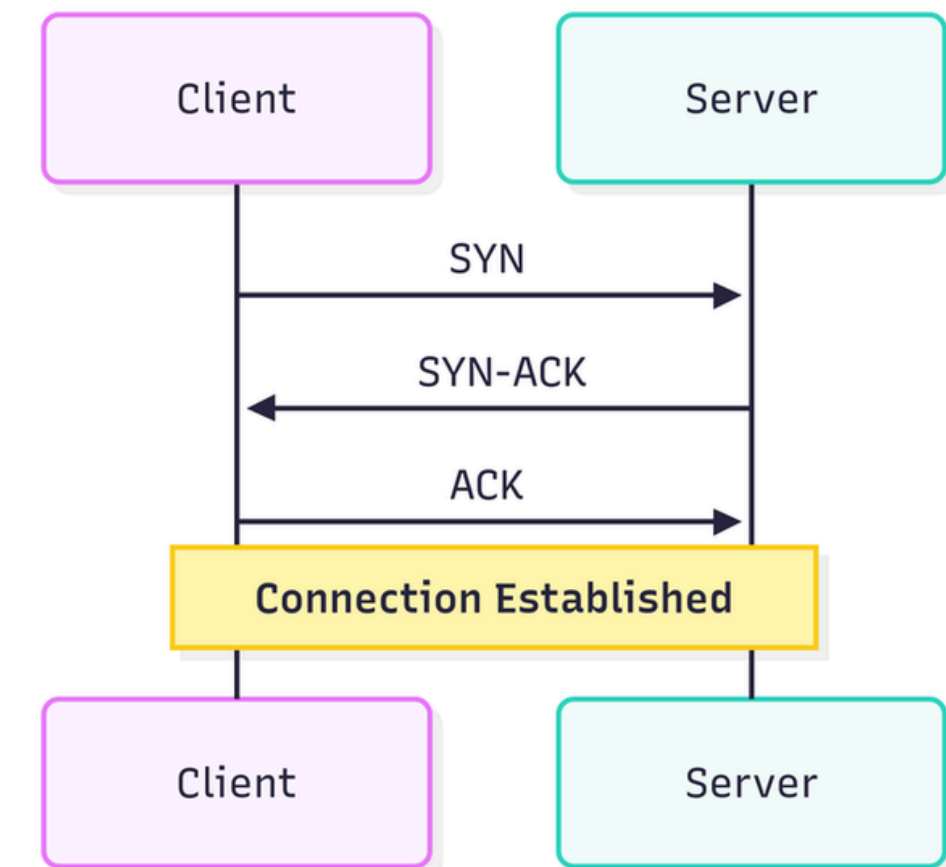
## Transport Layer



# **Building reliability from an unreliable channel**

# Transmission control protocol

- Establishes connection prior to data transmission.
- Maintains data integrity using sequence numbers, acknowledgments and checksums.
- Ensures reliable, ordered delivery of data packets, handling any loss.
- Handles the connection tear down process as well...



# TCP Connection Teardown

TCP has a polite "goodbye" process. It's usually a 4-Way Handshake.

**Step 1:** The "I have to go." (FIN)

*The client sends a FIN packet (for "finish")*

**Step 2:** The "Okay." (ACK)

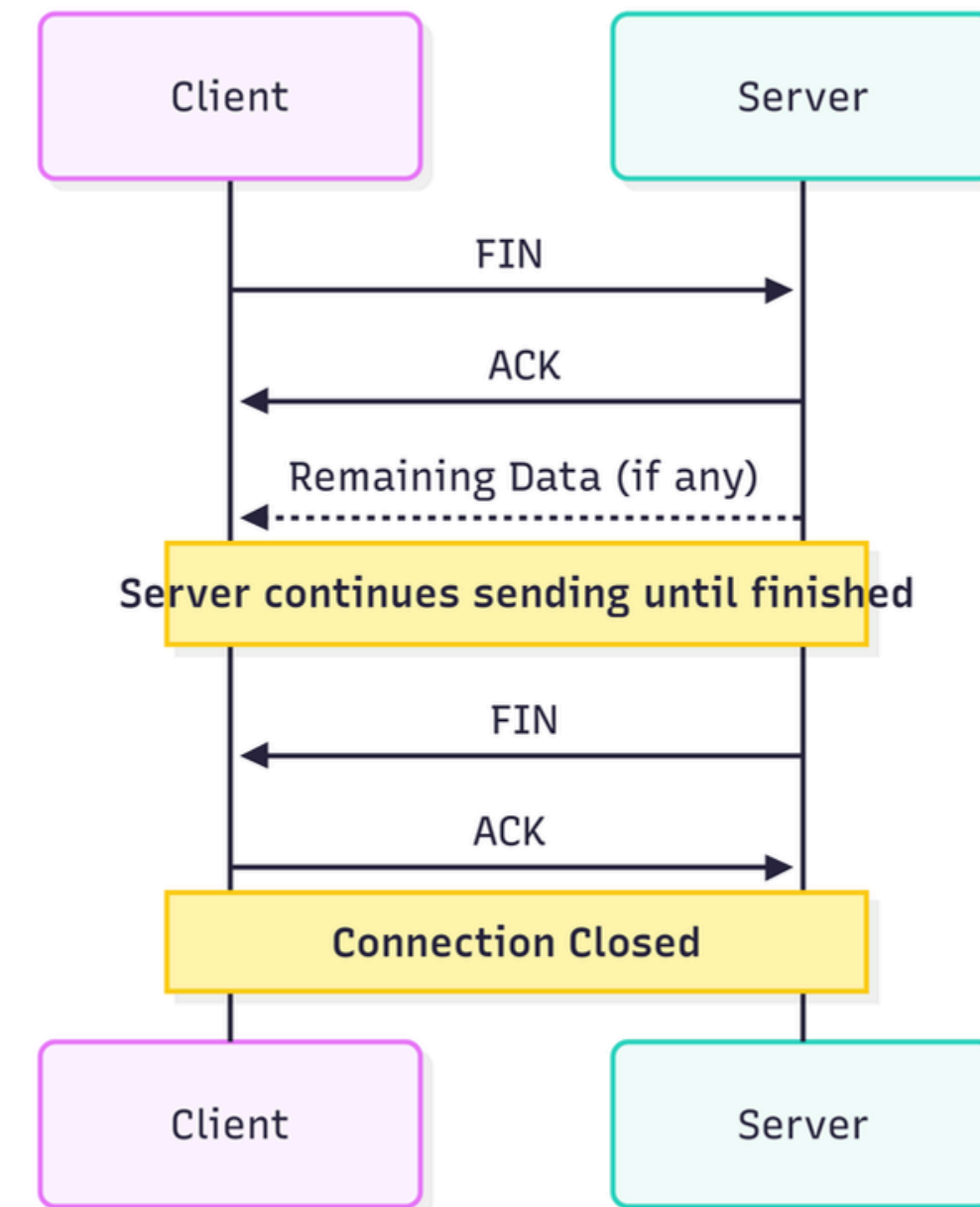
*The server receives this and acknowledges it.*

**Step 3:** The "I'm done too." (FIN)

*Once the server has finished sending any remaining data, it sends its own FIN packet.*

**Step 4:** The Final "Bye." (ACK)

*Your client gets the server's FIN and sends one last ACK.*



# Wireshark TCP echo server follow up

# Checksums:

As TCP aims to provide reliable data transfer, checksum detects transmission errors so that corrupted data can be retransmit.

**Let's imagine a tiny piece of data Alice wants to send, broken into two 8-bit chunks:**

**Chunk 1:** 10010011 (which is 147 in decimal)

**Chunk 2:** 01010110 (which is 86 in decimal)

**Step 1:** Sender's Side (Alice): TCP adds these two chunks together.

```
| 10010011
+ 01010110
-----
1 00001001 (There's a carry-over bit!)
```

# Checksums:

**Step 2:** TCP takes that carry-over 1 and adds it to the result.

$$\begin{array}{r} | \quad 00001001 \\ + \quad \quad \quad 1 \\ \hline | \quad 00001010 \quad (\text{This is the sum}) \end{array}$$

**Step 3:** Finally, it performs a "1's complement" (it flips all the bits) to create the checksum.  
So checksum becomes **11110101**.

# Checksums:

Alice sends the original data chunks and this checksum (11110101) to Bob.

$$\begin{array}{r} 10010011 \\ + 01010110 \\ \hline 00001010 \end{array}$$

Bob performs the same calculation on the data it received

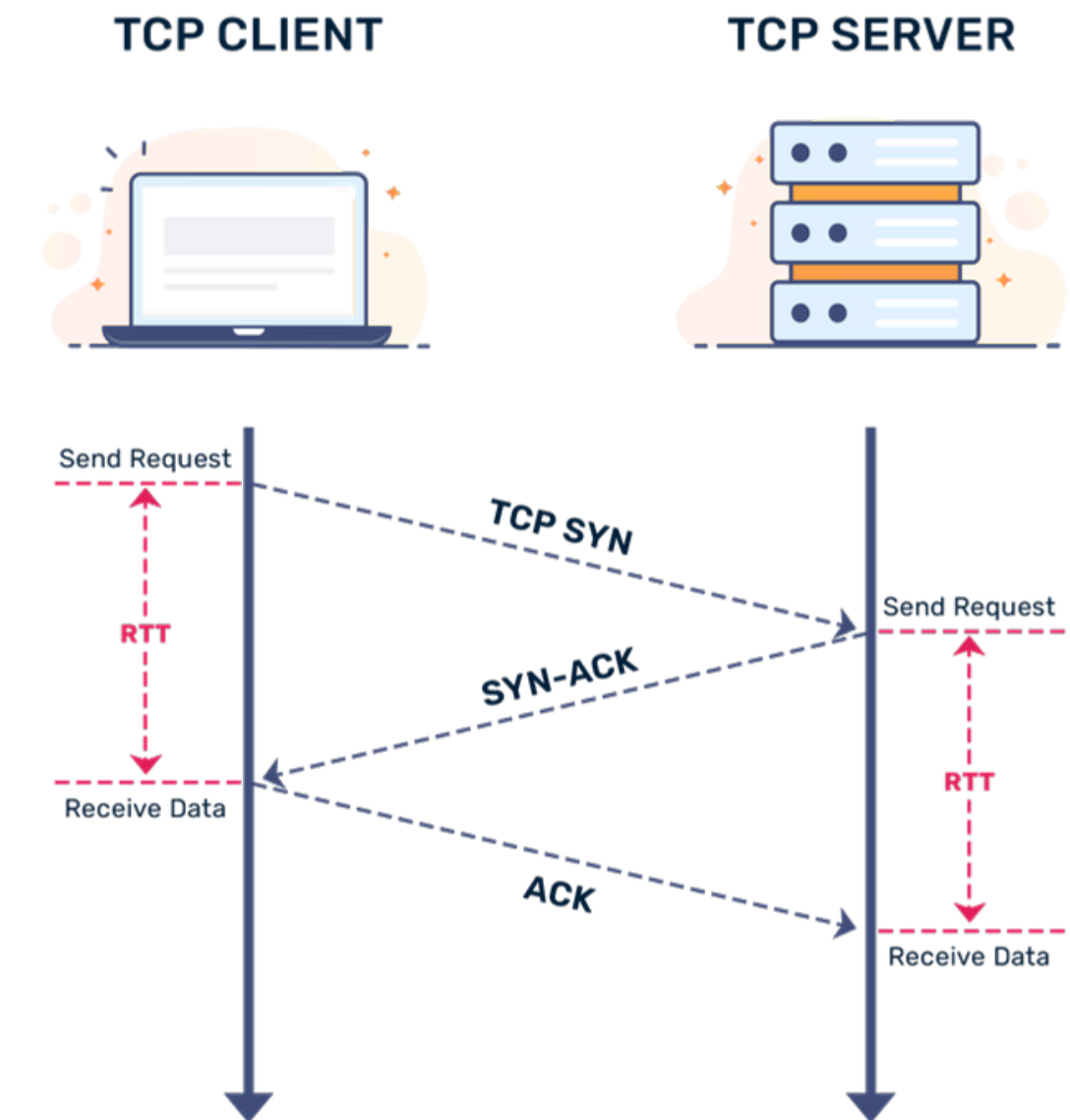
**And then adds the checksum it received.**

$$\begin{array}{r} 00001010 \text{ (Sum of the data)} \\ + 11110101 \text{ (Checksum from the data)} \\ \hline 11111111 \text{ (All 1s!)} \end{array}$$

***If the final result is all 1s, TCP knows with high probability that the data is perfect! ✓***

# The Waiting Game: Round Trip Time (RTT)

- Alice needs a receipt (an ACK, or Acknowledgment) from CAROL to know it arrived safely.
- How long should she wait before she assumes the page was lost? Ten milliseconds? A full second?
- This is where the Round Trip Time (RTT) comes in. It's the total time it takes for a segment to go from Alice to CAROL and for CAROL's acknowledgment to come back to Alice.



# Round Trip Time (RTT)

ping newtonschool.co

```
> ping newtonschool.co
PING newtonschool.co (99.83.190.102): 56 data bytes
64 bytes from 99.83.190.102: icmp_seq=0 ttl=249 time=24.607 ms
64 bytes from 99.83.190.102: icmp_seq=1 ttl=249 time=23.815 ms
64 bytes from 99.83.190.102: icmp_seq=2 ttl=249 time=24.538 ms
64 bytes from 99.83.190.102: icmp_seq=3 ttl=249 time=30.614 ms
^C
--- newtonschool.co ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 23.815/25.893/30.614/2.743 ms
```

ping taobao.com

```
kuldeepkrjha@Kuldeeps-MacBook-Pro ~ % ping taobao.com
PING taobao.com (59.82.121.163): 56 data bytes
64 bytes from 59.82.121.163: icmp_seq=0 ttl=79 time=429.367 ms
64 bytes from 59.82.121.163: icmp_seq=1 ttl=79 time=432.486 ms
64 bytes from 59.82.121.163: icmp_seq=2 ttl=79 time=478.576 ms
64 bytes from 59.82.121.163: icmp_seq=3 ttl=79 time=502.599 ms
64 bytes from 59.82.121.163: icmp_seq=4 ttl=79 time=523.929 ms
64 bytes from 59.82.121.163: icmp_seq=5 ttl=79 time=443.412 ms
64 bytes from 59.82.121.163: icmp_seq=6 ttl=79 time=461.422 ms
64 bytes from 59.82.121.163: icmp_seq=7 ttl=79 time=432.035 ms
64 bytes from 59.82.121.163: icmp_seq=8 ttl=79 time=428.674 ms
64 bytes from 59.82.121.163: icmp_seq=9 ttl=79 time=431.996 ms
64 bytes from 59.82.121.163: icmp_seq=10 ttl=79 time=430.521 ms
64 bytes from 59.82.121.163: icmp_seq=11 ttl=79 time=434.289 ms
64 bytes from 59.82.121.163: icmp_seq=12 ttl=79 time=430.696 ms
64 bytes from 59.82.121.163: icmp_seq=13 ttl=79 time=428.899 ms
64 bytes from 59.82.121.163: icmp_seq=14 ttl=79 time=431.847 ms
64 bytes from 59.82.121.163: icmp_seq=15 ttl=79 time=429.329 ms
64 bytes from 59.82.121.163: icmp_seq=16 ttl=79 time=440.055 ms
64 bytes from 59.82.121.163: icmp_seq=17 ttl=79 time=464.146 ms
64 bytes from 59.82.121.163: icmp_seq=18 ttl=79 time=433.413 ms
^C
--- taobao.com ping statistics ---
19 packets transmitted, 19 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 428.674/446.721/523.929/26.781 ms
kuldeepkrjha@Kuldeeps-MacBook-Pro ~ %
```

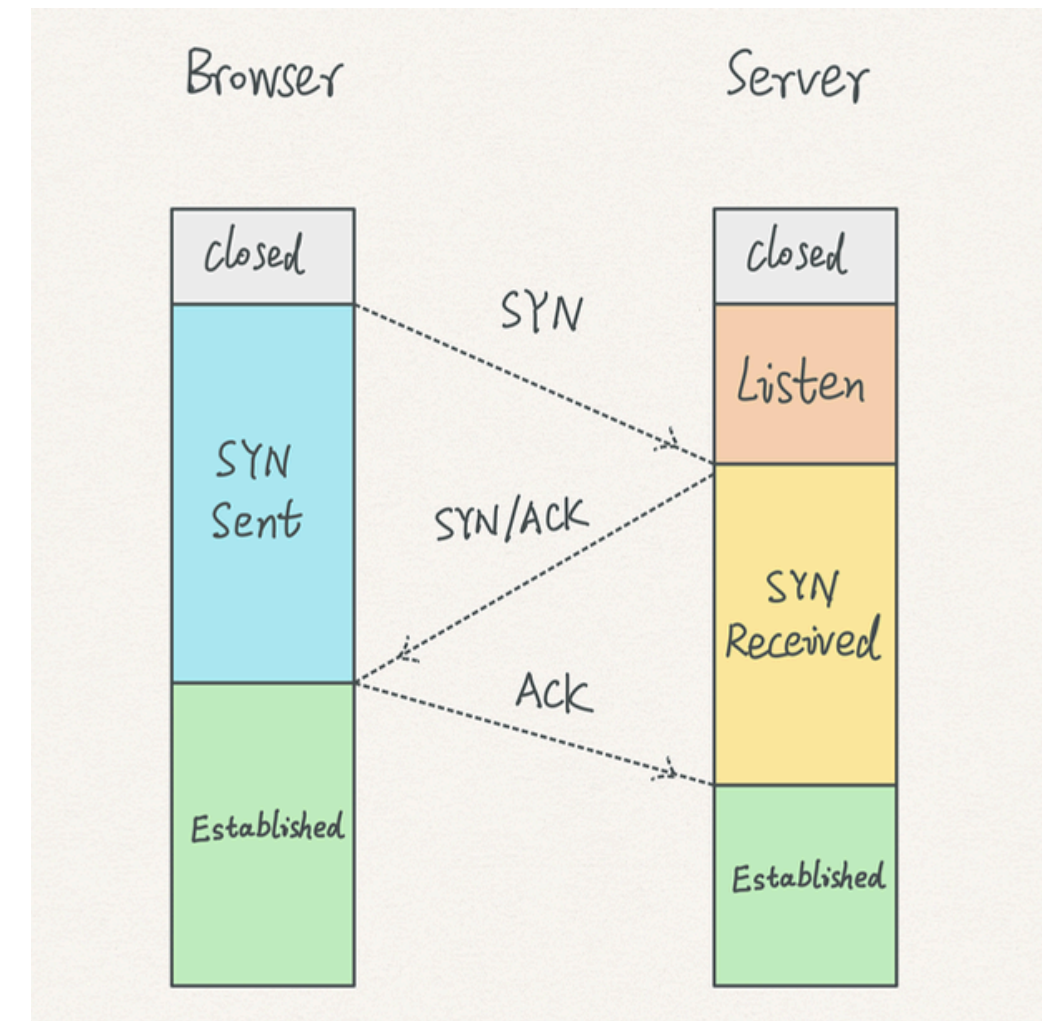
# **Stop and Wait Protocol**

## **Sliding Window Protocol**

# Why is Flow Control Needed?

- Not all devices have the same buffer size or processing power.
- A fast sender could flood a slow receiver, leading to packet loss or buffer overflow.
- TCP flow control prevents this by allowing the receiver to say:

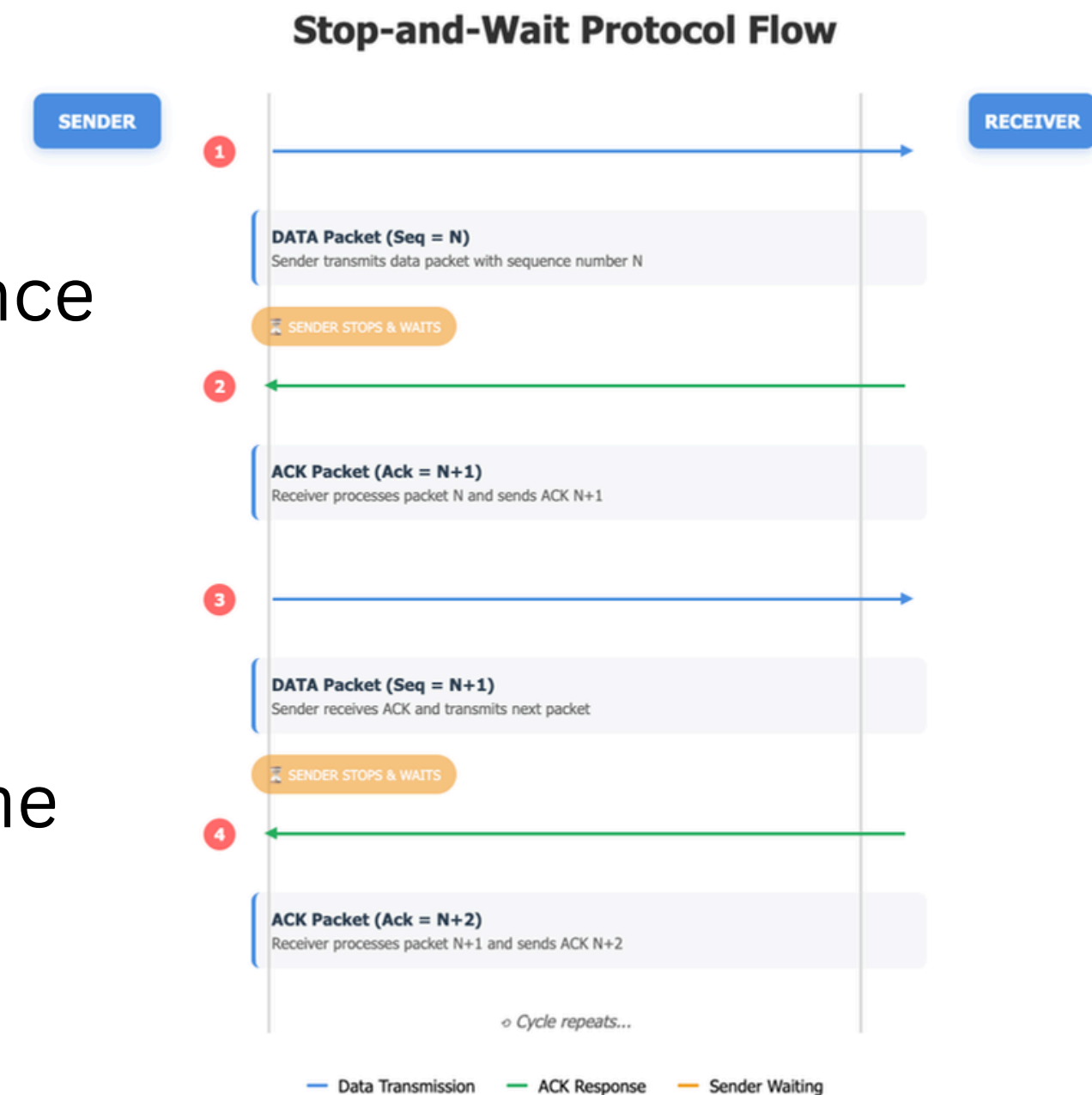
“Hey, slow down — I can only handle this much right now.”



# How Stop-and-Wait Works

<https://nstclassdemo2025.pages.dev/stop-and-wait>

- **Sender:** Transmits a data packet with sequence number N
- **Sender:** Stops all transmission and waits
- **Receiver:** Receives packet N, processes it
- **Receiver:** Sends ACK with acknowledgment number N+1 (indicating "I received N, send me N+1 next")



# Stop & Wait Mechanism

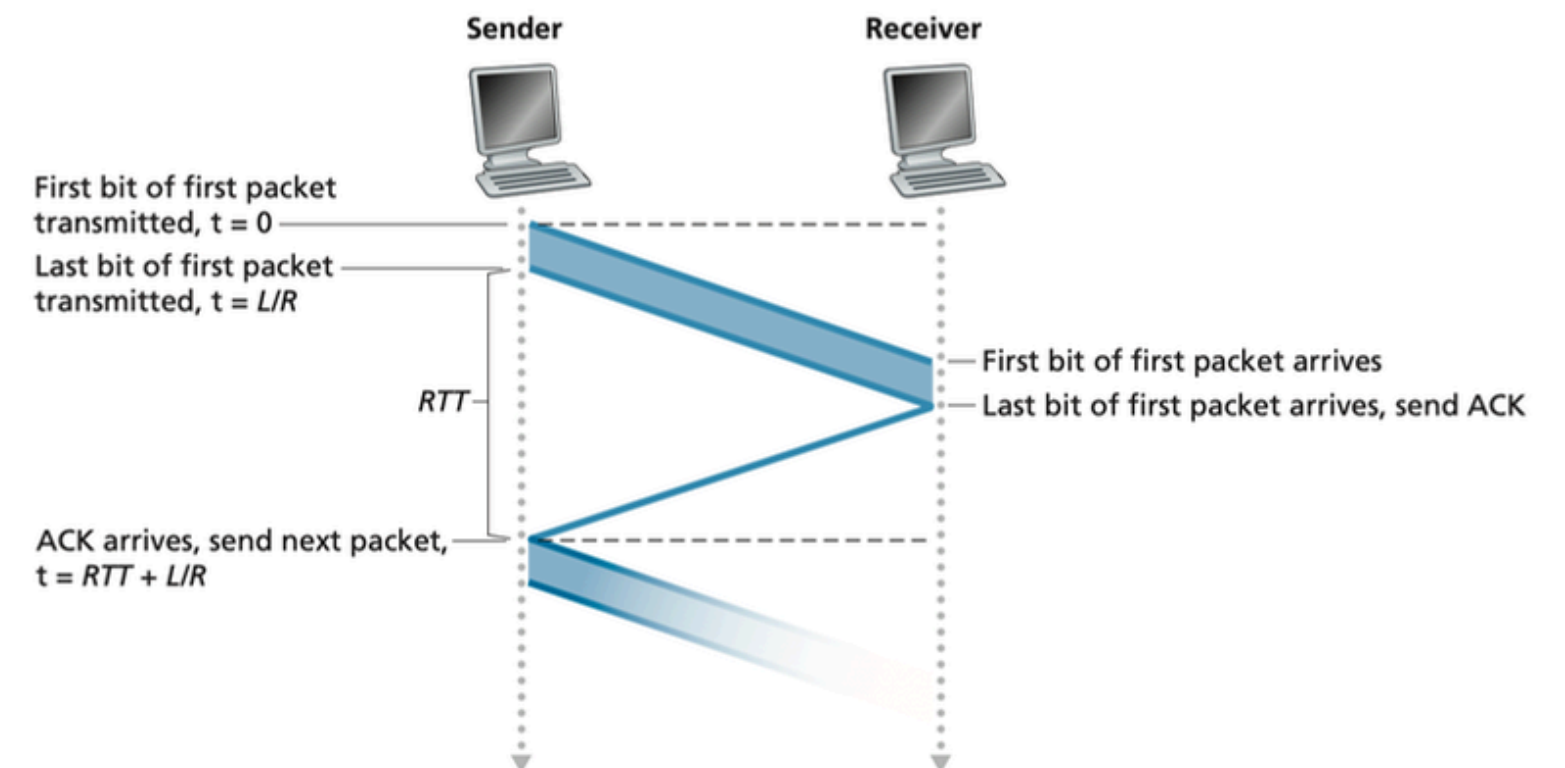
- Very inefficient for high-latency or high-bandwidth networks.
- Example: On a long-distance link, the sender wastes time waiting for each ACK before sending again.

Utilization is low:

$$\text{Utilization} = \frac{T_{frame}}{T_{frame} + 2 \cdot T_{prop}}$$

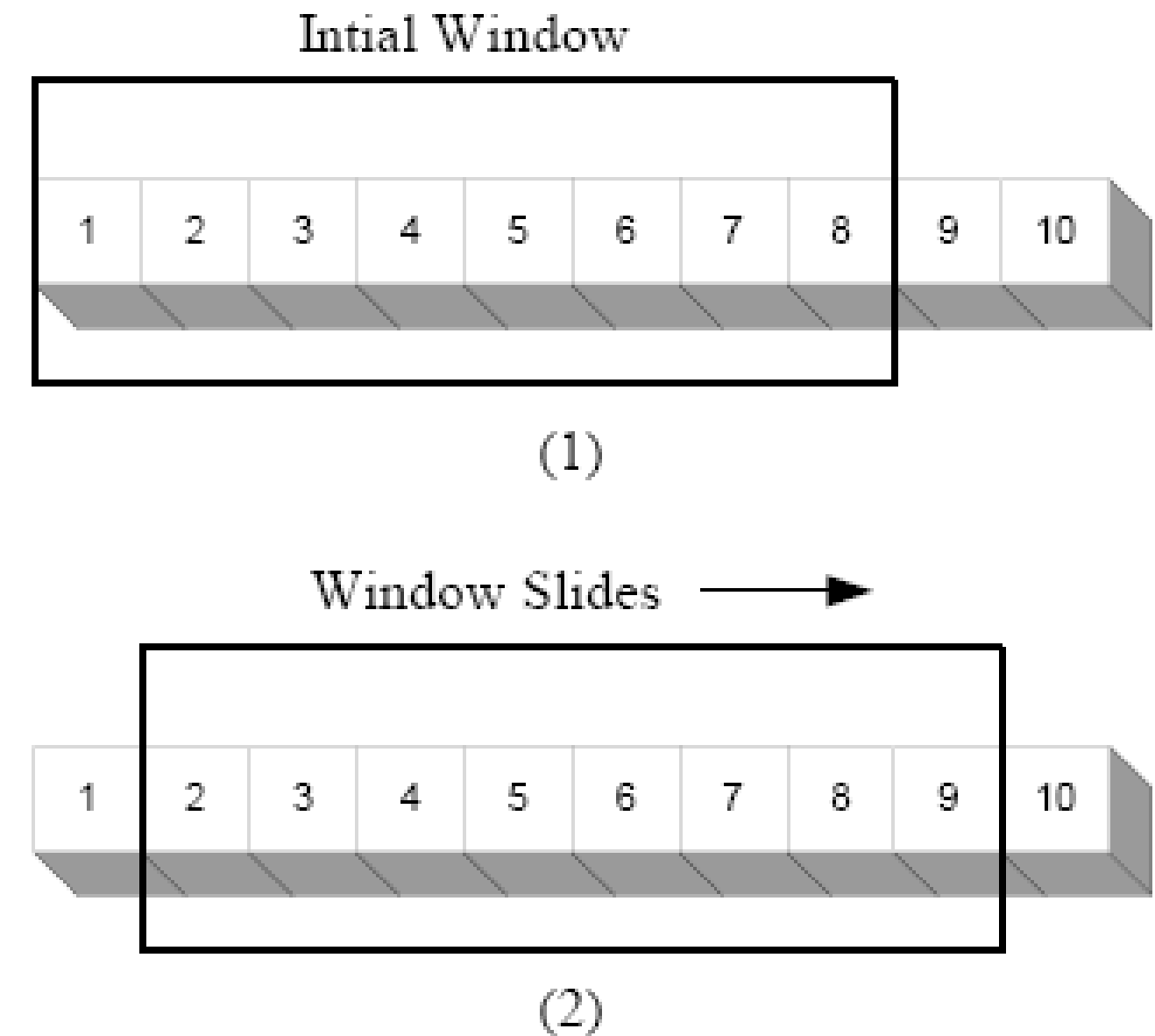
$T_{frame} \rightarrow$  transmission time

$T_{prop} \rightarrow$  propagation delay



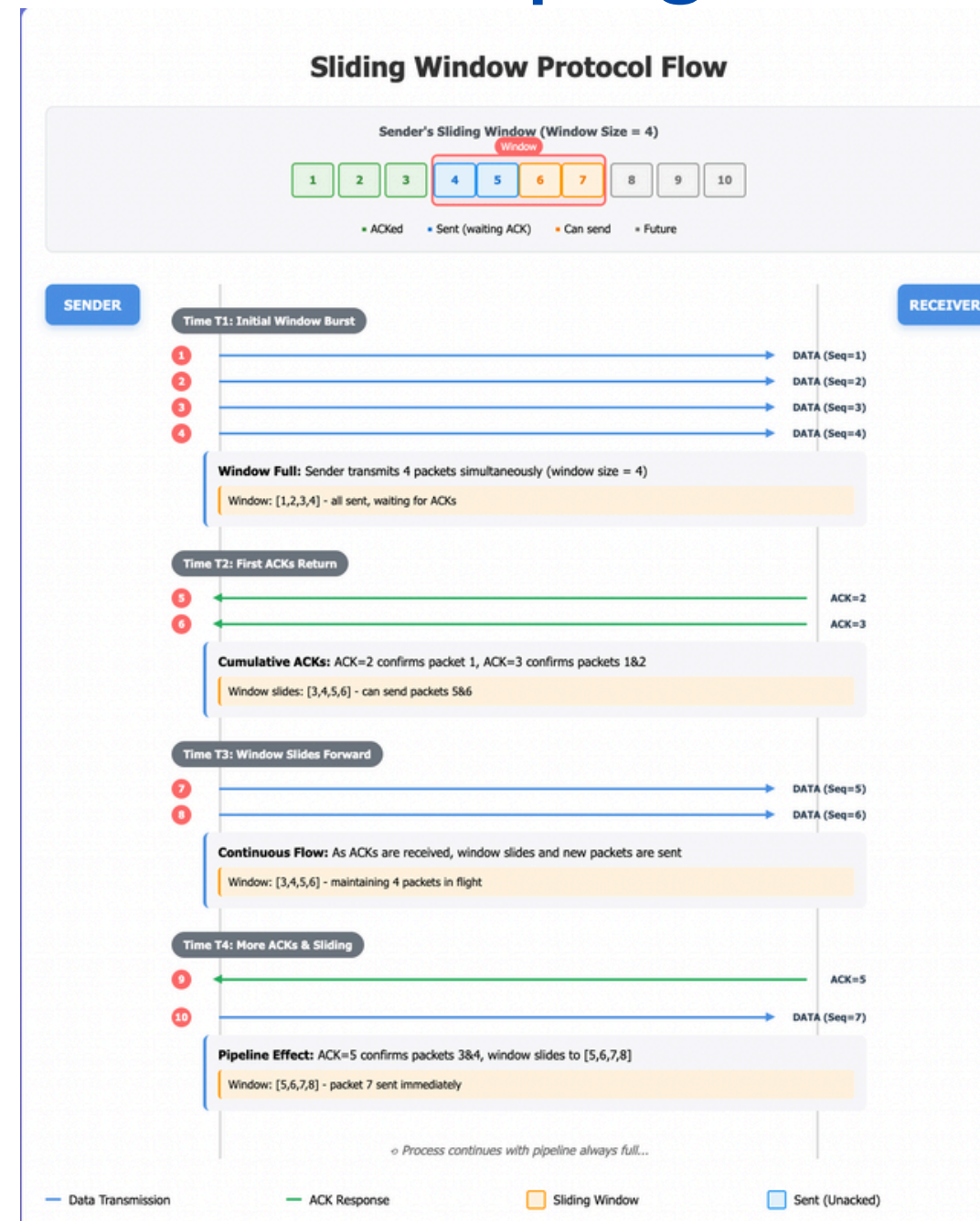
# Sliding Window Protocol

- TCP uses a "sliding window protocol" to manage flow control.
- The receiver tells the sender how much data it can receive at a time (called the window size).
- As the receiver processes data and acknowledges packets, the sender "slides" the window forward and sends more.



# How Sliding Window Protocol Works

<https://nstclassdemo2025.pages.dev/sliding-window>



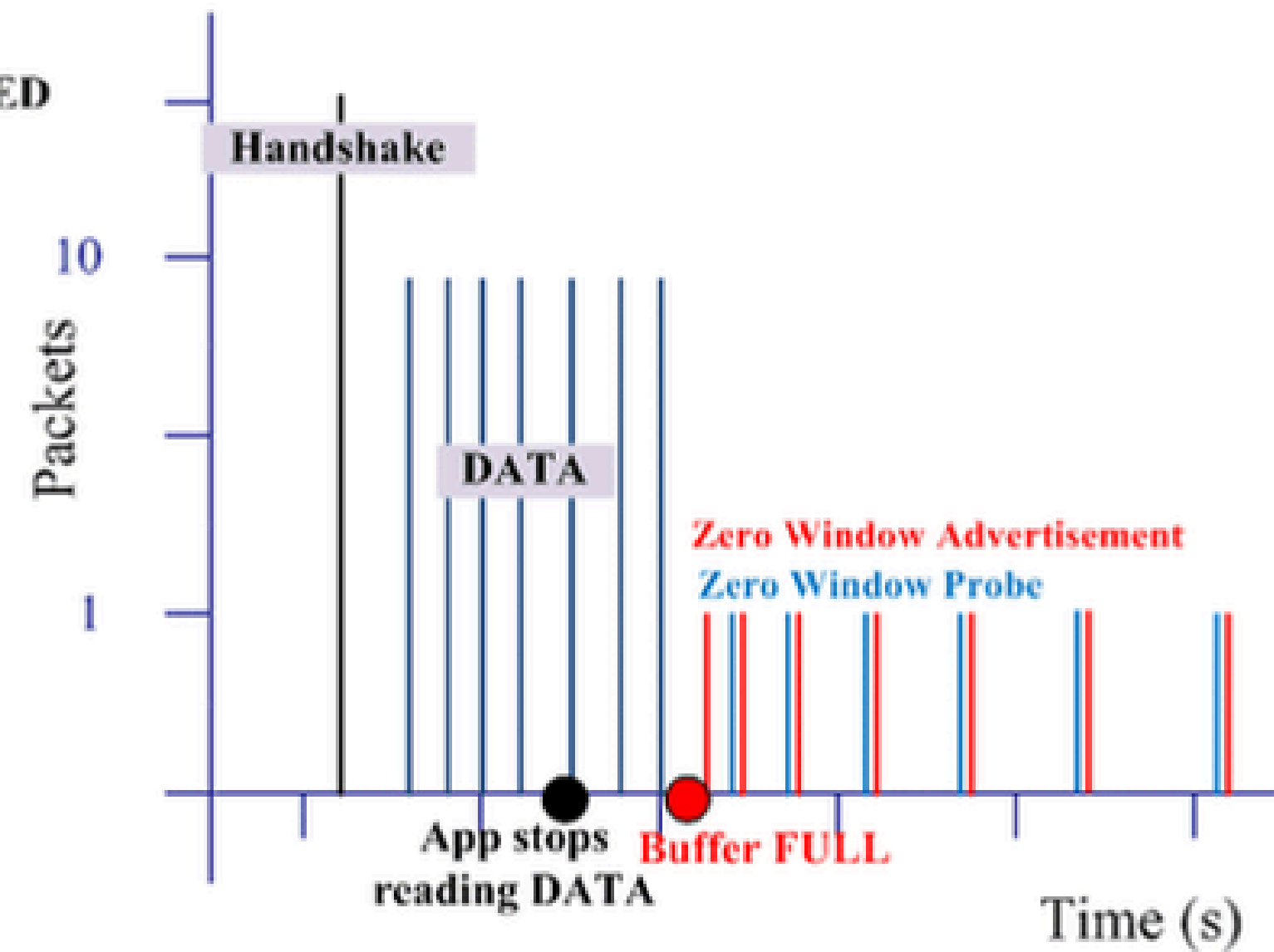
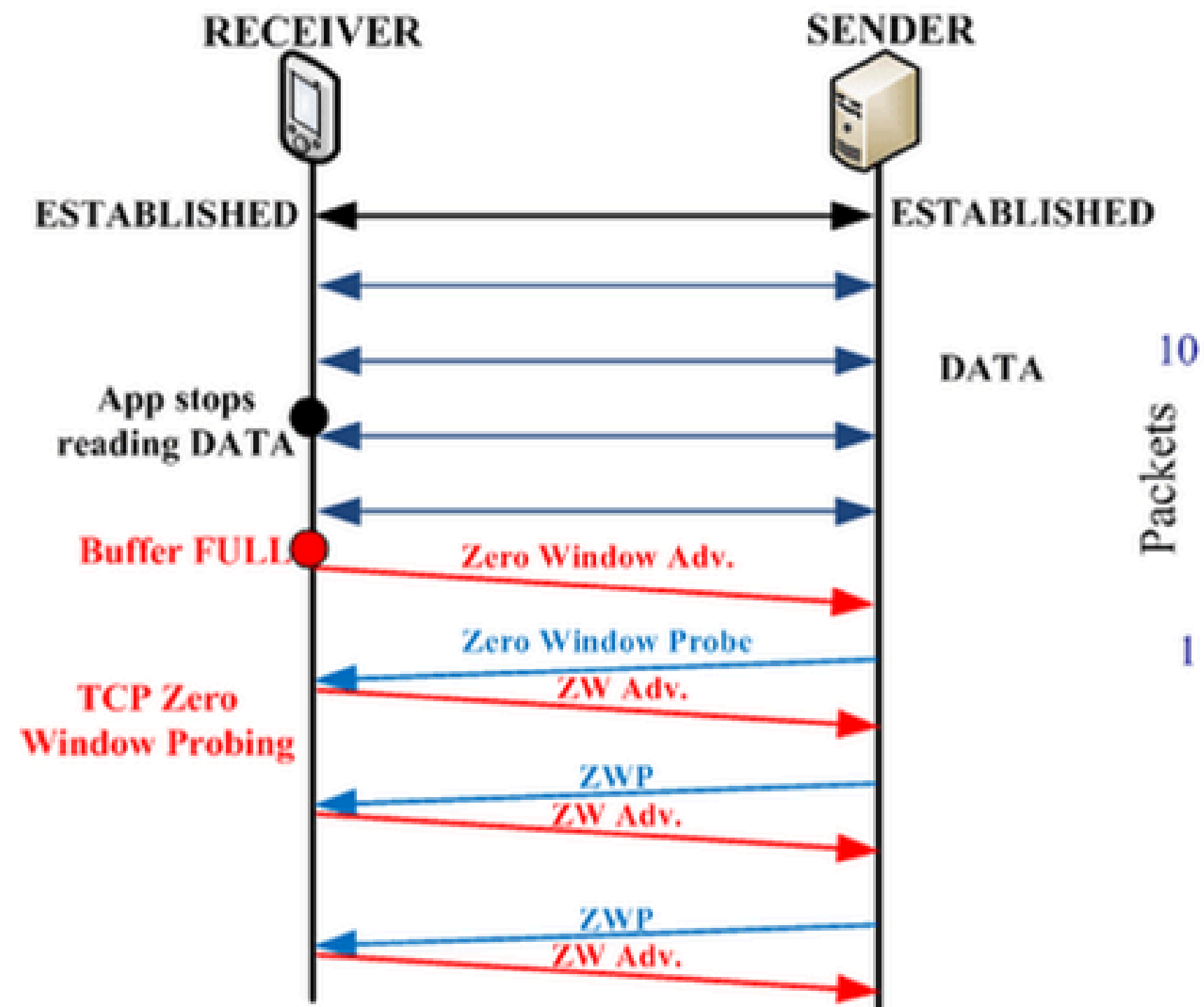
# Zero Window Condition

A Zero Window condition occurs in TCP flow control when the receiver's buffer is full, and it temporarily tells the sender to stop sending data.

This is done by the receiver advertising a window size of zero in the TCP header

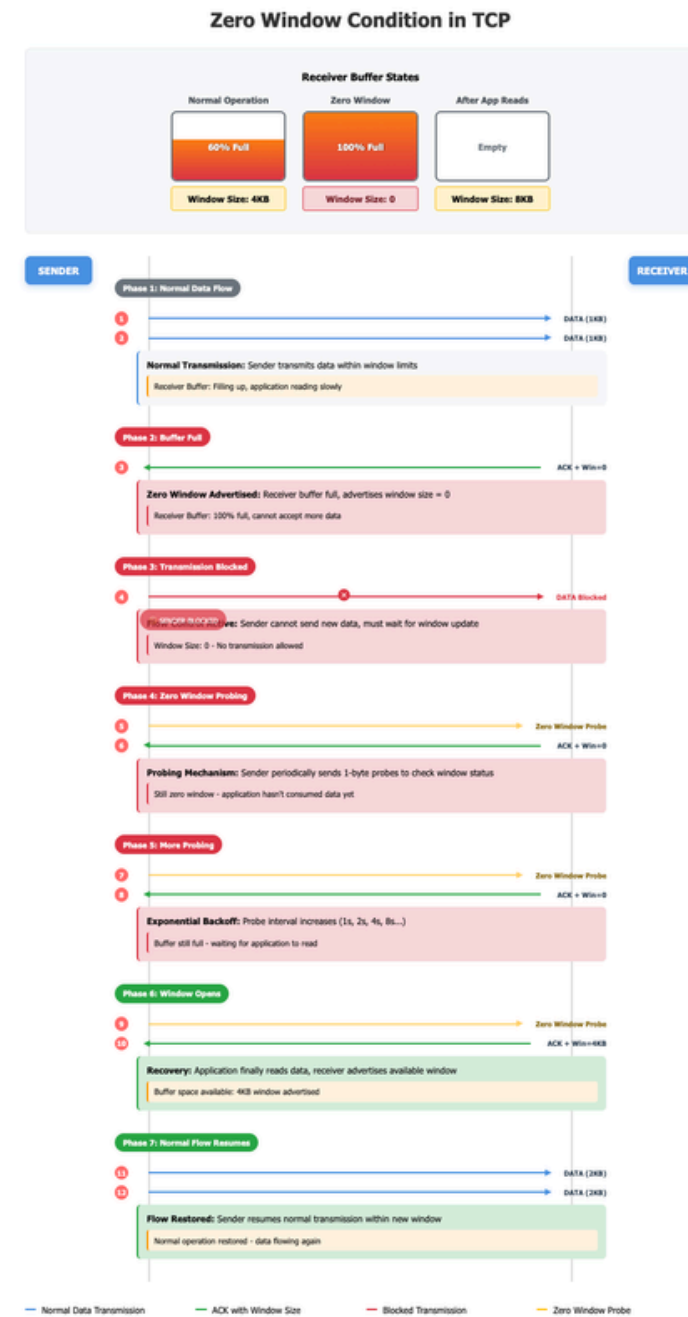


# Zero Window Condition



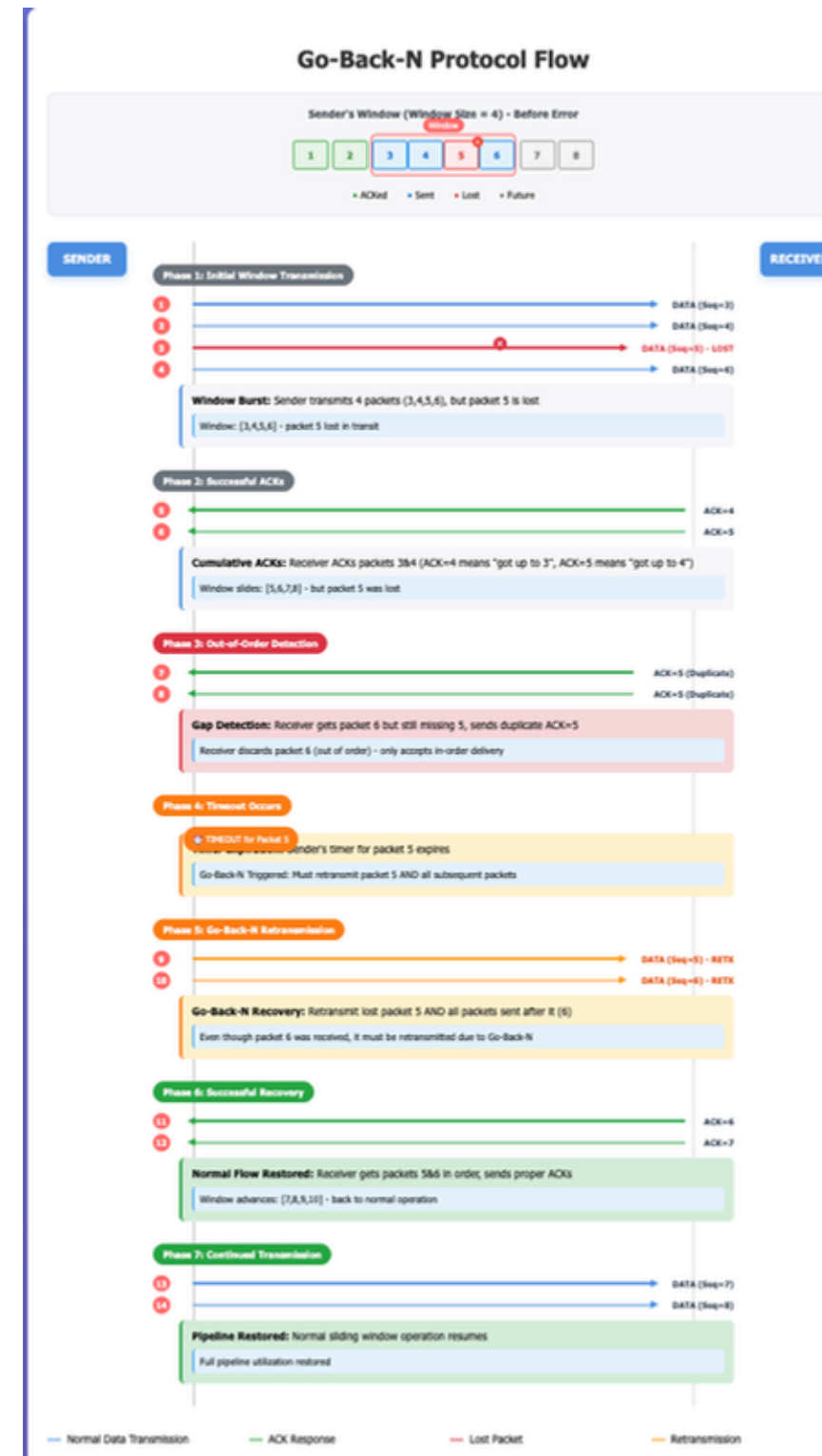
# Zero Window Condition

<https://nstclassdemo2025.pages.dev/zero-window>



# Go-Back-N

- Go-Back-N (GBN) is an error control protocol designed for reliable communication over noisy channels.
- It enables the sender to send multiple frames before receiving an acknowledgment; however, if a frame is lost or erroneous, the sender must retransmit that frame and all subsequent frames.



<https://computerscience.unicam.it/marcantoni/reti/applet/GoBackProtocol/goback.html>

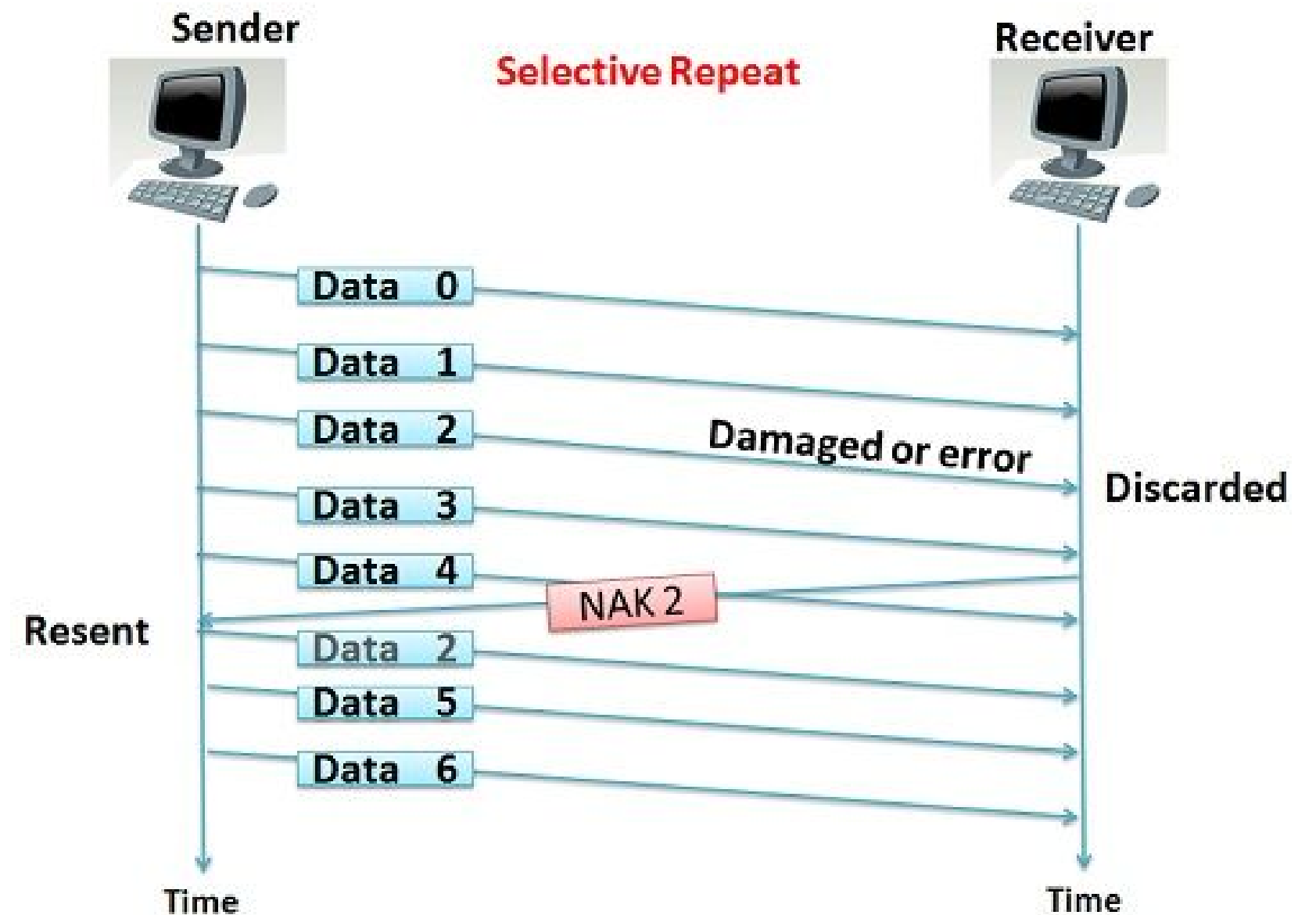
# How Go-Back-N Works

- Sender sends N frames at once without waiting for individual ACKs.
- Receiver only accepts frames in order (i.e., sequentially).
- If a frame is lost or arrives corrupted, the receiver:  

Discards that frame and all that follow, even if they're correct.
- The sender goes back to the last acknowledged frame and retransmits all frames from there.

# Selective Repeat

- Selective Repeat (SR) is an error-control protocol used in data transmission to ensure reliable delivery over unreliable or noisy networks.
- Unlike Go-Back-N, it only retransmits the frames that were lost or corrupted, not all subsequent frames.



# How Selective Repeat Works

The sender can send multiple frames up to a specified window size without waiting for ACKs.

- The receiver can accept and buffer frames out of order.
- If a frame is lost or corrupted:

The receiver discards only that frame.

Sends a Negative ACK (NACK) or simply doesn't ACK that frame.

- The sender retransmits only the missing frame, not the whole window.

[https://media.pearsoncmg.com/ph/esm/ecs\\_kurose\\_compnetwork\\_8/cw/content/interactiveanimations/selective-repeat-protocol/index.html](https://media.pearsoncmg.com/ph/esm/ecs_kurose_compnetwork_8/cw/content/interactiveanimations/selective-repeat-protocol/index.html)

**Please fill the feedback form**

**Thank You**